

Information Processing 1 — Week 10 exercises

Complete each exercise on your own laptop before the end of the exercise class. When you finish an exercise, **immediately** ask an instructor to check your answer. Answers checked after the end of the class will have a 50% penalty.

Do not struggle for more than a few minutes with anything you cannot solve. If you are stuck or have any questions then **raise your hand** and an instructor will help you.

1 [5 points] Processing data in lists

Last week we processed the contents of a text file as a sequence of lines read one at a time. This week we will process a sequence of items that is kept in memory, finishing by reading an entire file into memory so that its contents can be tested many times without re-reading it. A list is the perfect data type for holding this kind of data in memory.

1.1 Sum of integers in a list

Write a function `total(numbers)` that returns the sum of all the numbers in the list `numbers`. Test your function on several different lists.

```
print(total([1, 1, 1, 1, 1])) # 5
print(total([1, 2, 3, 4, 5])) # 15
print(total(range(10)))      # 45
```

1.2 Cumulative sum of integers in a list

Make a copy of your function and call it `cumulative(numbers)`. Modify it so that it returns a list of cumulative sums of the `numbers`. For example, the 3rd element of the result should be the cumulative sum of the first 3 elements in `numbers`. Test your function on several different lists.

```
print(cumulative([]))          # []
print(cumulative([1]))        # [1]
print(cumulative([1, 2, 3, 4, 5])) # [1, 3, 6, 10, 15]
print(cumulative(range(10)))   # [0, 1, 3, 6, 10, 15, 21, 28, 36, 45]
```

1.3 Sum of integers in nested lists

Lists can contain lists. Processing each element in nested lists is easy to do with a recursive function.

Write a function `nested_sum(n)` that returns the sum of the numbers in the list `n` and any nested sub-lists that `n` might contain (to any depth of nesting). Beware that `n` could also be a single number, in which case just return it as the result.

To test whether `n` is an integer or a nested list you can use the unary operator `'type(x)'`, which tells you the type of the value `x`, and the binary operator `'is'`, which is `True` if the left and right hand sides are the same thing. For example:

```

if type(n) is int:
    # do something with the integer value in n
elif type(n) is list:
    # do something with the list value in n
else:
    # do something with a value that is neither integer nor list

```

Test your function on some suitably nested list, for example:

```
print(nested_sum([None, 1, [2, "two", [[[3, False, 4]]]], [], 5)))
```

1.4 Testing the order of elements in a list

Write a function called `is_ordered(things)` that returns `True` if the elements in the sequence `things` are all in ascending order (according to `<=`) and `False` if not. Test your function on several different lists of things.

```

print(isordered([]))           # True
print(isordered([1, 1, 1]))    # True
print(isordered([1, 2, 3]))    # True
print(isordered([1, 3, 2]))    # False
print(isordered([2, 1, 3]))    # False
print(isordered([[1],[2]]))    # True
print(isordered([[2],[1]]))    # False
print(isordered(["any", "body", "can", "dance", "even", "fred"]))    # True
#print(isordered(["any", 1, "can", "dance"]))    # Error!

```

1.5 Detecting anagrams

Two words are *anagrams* of each other if you can rearrange the letters of one of them to make the other.

Write a function `is_anagram(a, b)` that returns `True` if `a` is an anagram of `b`.

For example, `is_anagram("cat", "act")` should be `True` but `is_anagram("bat", "bad")` should be `False`. Test your function on several pairs of words.

```

print(is_anagram("tone", "note"))    # True
print(is_anagram("chemical", "alchemic"))    # True
print(is_anagram("detail", "dilate"))    # True
print(is_anagram("angered", "enraged"))    # True
print(is_anagram("tangled", "tingled"))    # False
print(is_anagram("goat", "boat"))    # False

```

Hint: This function is *really* easy to write if you can think of an operation that will transform `a` and `b` into identical values if they are anagrams of each other.

▷▷ Ask an instructor to check and record your work now (before you run out of time).

2 [3 points] Simulation as an experimental method

Simulation allows you to perform an experiment virtually, without having to perform it in the real world. This lets you perform an impractical experiment many times and obtain a statistical result by looking at

your results. The next three exercises lead you through the creation of a simulation that answers the question, “How many randomly selected people do you need to put in a room to have a 50% chance that two of them share the same birthday?”

2.1 Building a list of random numbers

The function `x.append(y)` adds the element `y` to the end of the list `x`. The function `randint(i, j)` from the module `random` generates a random integer between `i` and `j`, inclusive.

```
from random import randint
print(randint(1, 10)) # a random integer in the range [1 ... 10]
```

Write a function `randlist(n, i, j)` that creates a list of `n` random integers in the range `i` to `j`, inclusive. Test your function by running

```
print(randlist(10, 1, 5))
```

many times and checking you get a list of 10 random integers that often include 1 and 5.

2.2 Detecting duplicated values

Write a function `has_duplicates(l)` that returns `True` if the list `l` has any duplicated elements (i.e., the same value occurs more than once in `l`). For example, `has_duplicates([1, 2, 3])` returns `False`, but `has_duplicates([1, 2, 3, 1])` returns `True`. Test your function on several different lists.

```
print(has_duplicates([1, 2, 3])) # False
print(has_duplicates([1, 1, 2, 3])) # True
print(has_duplicates([1, 2, 2, 3])) # True
print(has_duplicates([1, 2, 3, 3])) # True
print(has_duplicates([1, 2, 3, 1])) # True
```

Hint: If a list contains duplicates then a sorted version of the same list will contain at least one consecutive pair of elements that are the same. Testing for that is far more efficient than testing each element in turn to see if it occurs elsewhere in the list.

2.3 Statistical simulation

You can number the days of the year from 1 to 365. Using `randlist()` you can create a list of `n` random days of the year.

Write a program that generates 100,000 lists of `n` random days of the year and counts how many of those lists contain duplicated values. Convert that count into a percentage.

What is the minimum value of `n` that consistently gives you a result of at least 50%?

How many random people do you need to put in a room to have, on average, a 50% chance that two of them share the same birthday?

How many random people do you need to put in a room to have, on average, a 90% chance that two of them share the same birthday?

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

3 [2 points] Big data: searching a very large collection

Last week we used a file containing 113,783 words from the English language. Searching that file for certain words can be very expensive.

We can exploit the fact that the words are in order to search them very quickly. To motivate this, let's find pairs of words in that file where one word contains the same letters as the other but in the opposite order.

3.1 Reading data from a file into an list

Write a program that reads the contents of `words.txt` into an list called `words`. Be sure to `strip()` the whitespace from the ends of every word in the list.

3.2 Finding reverse pairs

Two words are a 'reverse pair' if one of them is the same as the other except with the letters backwards. For example, 'pool' and 'loop' are a reverse pair. Since you can use the slice operator '`[ : : ]`' to reverse a sequence, testing for a reverse pair is trivial.

Write a program that prints every reverse pair of words in the `words` list of the previous exercise.

Hint: The expression '`x in y`' searches for `x` in `y` and returns `True` if it is found.

▷▷ Ask an instructor to check and record your work now (before you run out of time).

3.3 [1 bonus point] Making word search more efficient

Your program in the previous exercise probably runs very slowly. That is because it is searching for each reversed word in `words` by checking every word, from the first to the last.

We can do a lot better than that, because `words` is sorted into alphabetical order.

The following algorithm describes *binary search*, which very quickly finds a `target` in some `sequence`, provided `sequence` is sorted.

1. let `first` be the index of the first element in `sequence`
2. let `last` be the index of the last element in `sequence`
3. while the range to search is not empty, i.e., `first <= last`:
 - (a) let `mid` be the index mid-way between `first` and `last`, i.e., `(first+last)//2`
 - (b) let `elt` be the element at that index
 - (c) if `target` is `<` than `elt`, discard the right half of the range by setting `last = mid - 1`
 - (d) if `target` is `>` than `elt`, discard the left half of the range by setting `first = mid + 1`
 - (e) if `target` and `elt` are the same, return success
4. return failure

This works by repeatedly dividing the range of elements to search (between `first` and `last`) in half, discarding the half that does not contain the `target`. To do this it compares the element halfway between `first` and `last` with `target`. Eventually either `target` is found, or the size of the range drops to zero meaning that `target` is not in the `sequence`.

Implement a function called `includes(sequence, target)` that uses binary search to find the `target` element in the given `sequence`. Return `True` if it is found, otherwise `False`.

Modify your program from the previous exercise to use `includes(words, revword)` instead of `revword in words` to check if the reversed version of each word is in the `words`.

How much faster does your program run now?

▷▷ Ask an instructor to check and record your work now (before you run out of time).

4 Challenges

Use your very fast `includes(sequence, target)` function to search for some more interesting words in `words`.

4.1 Interlocking words

Two words `a` and `b` ‘interlock’ when you can form a new, valid word by taking a letter alternately from `a` and `b`. For example, ‘shoe’ and ‘cold’ interlock because if you take letters from each alternately you obtain the word ‘schooled’, which is a valid word present in `words`.

Write a program that finds and prints all pairs of words in `words` that interlock.

Hints: Don’t enumerate all possible pairs of words! Instead think more cleverly about the problem. For each word in `words`, split it into two smaller words `a` and `b` where `a` contains the first, third, fifth, etc., letter from the word and `b` contains the second, fourth, sixth, etc., letter. (This is really easy because the slice operator ‘`[: :]`’ can extract these letters for you.) For example, split ‘schooled’ into ‘shoe’ and ‘cold’ and then search for the two shorter words in `words`.

You can ignore trivial interlocking words that are less than three letters long (which means you don’t need to consider splitting original words that are less than 6 characters long). Use your search function to search `words` for each of your two split words. (Your program will run very slowly if you use the ‘`in`’ operator.)

How many pairs of interlocking words of three or more letters are there in `words`?

4.2 Three-way interlocking words

Find all the three-way interlocking words. These are words in which every third letter forms a word, starting at the first, second, and third letter of the original word. Again, only consider candidates that can be split into three smaller words of three letters or more.

How many triples of interlocking words of three or more letters are there in `words`?